

Libraries Import

```
In [69]: # Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import roc_auc_score, roc_curve, mean_squared_error, r2_score

# Statistics
from scipy import stats
from scipy.stats import iqr

# Settings
import warnings
warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-darkgrid')

print("All libraries imported successfully!")
```

All libraries imported successfully!

Dataset Generate

```
In [65]: # Dataset
np.random.seed(42)
n_samples = 10000

data = {
    'CustomerID': range(1, n_samples + 1),
    'Age': np.random.randint(18, 65, n_samples),
    'Gender': np.random.choice(['Male', 'Female'], n_samples),
    'Annual_Income': np.random.randint(20000, 150000, n_samples),
    'Num_Website_Visits': np.random.randint(1, 20, n_samples),
    'Time_Spent_Minutes': np.random.randint(5, 120, n_samples),
    'Pages_Viewed': np.random.randint(1, 50, n_samples),
    'Items_Viewed': np.random.randint(1, 30, n_samples),
    'Cart_Additions': np.random.randint(0, 15, n_samples),
    'Previous_Purchases': np.random.randint(0, 10, n_samples),
    'Days_Since_Last_Visit': np.random.randint(1, 365, n_samples),
    'Discount_Sensitive': np.random.choice([0, 1], n_samples, p=[0.6, 0.4]),
    'Device_Type': np.random.choice(['Mobile', 'Desktop', 'Tablet'], n_samples),
    'Browser': np.random.choice(['Chrome', 'Firefox', 'Safari', 'Edge'], n_samples)
```

```

    'Season': np.random.choice(['Winter', 'Spring', 'Summer', 'Fall'], n_samples),
    'Is_Weekend': np.random.choice([0, 1], n_samples, p=[0.7, 0.3]),
    'Customer_Satisfaction': np.random.randint(1, 6, n_samples),
}

df = pd.DataFrame(data)

# Purchase probability
purchase_prob = (
    0.3 * (df['Time_Spent_Minutes'] / df['Time_Spent_Minutes'].max()) +
    0.2 * (df['Pages_Viewed'] / df['Pages_Viewed'].max()) +
    0.2 * (df['Cart_Additions'] / 15) +
    0.15 * (df['Previous_Purchases'] / 10) +
    0.1 * (df['Discount_Sensitive']) +
    0.05 * (df['Is_Weekend'])
)
purchase_prob += np.random.normal(0, 0.1, n_samples)
purchase_prob = np.clip(purchase_prob, 0, 1)

df['Purchase'] = np.random.binomial(1, purchase_prob)
df['Purchase_Amount'] = np.where(df['Purchase'] == 1, np.random.randint(10, 500, n_

print(f"Dataset ready! {len(df)} rows")
print(f"Purchase rate: {df['Purchase'].mean():.2%}")

```

Dataset ready! 10000 rows

Purchase rate: 46.83%

Data Cleaning

```

In [53]: # Missing values
np.random.seed(42)
missing_idx = np.random.choice(df.index, size=50, replace=False)
df.loc[missing_idx, 'Customer_Satisfaction'] = np.nan

# Handle missing values
df['Customer_Satisfaction'].fillna(df['Customer_Satisfaction'].median(), inplace=True)

# Data types
df['Gender'] = df['Gender'].astype('category')
df['Device_Type'] = df['Device_Type'].astype('category')
df['CustomerID'] = df['CustomerID'].astype(str)

print("Data cleaned!")
print(f"Missing values: {df.isnull().sum().sum()}")

```

Data cleaned!

Missing values: 50

Numpy Operations

```

In [54]: print("=" * 50)
print("NUMPY OPERATIONS")

```

```

print("=" * 50)

np_array = df[['Age', 'Annual_Income', 'Time_Spent_Minutes']].values
print(f"Array shape: {np_array.shape}")

print(f"\nMean Age: {np.mean(np_array[:, 0]):.2f}")
print(f"Median Income: {np.median(np_array[:, 1]):.2f}")
print(f"Std Time: {np.std(np_array[:, 2]):.2f}")

correlation = np.corrcoef(np_array[:, 0], np_array[:, 1])[0,1]
print(f"Age-Income Correlation: {correlation:.3f}")

```

```

=====
NUMPY OPERATIONS
=====
Array shape: (10000, 3)

Mean Age: 41.05
Median Income: 85182.00
Std Time: 33.06
Age-Income Correlation: 0.015

```

Pandas Transformation

```

In [55]: print("=" * 50)
print("PANDAS TRANSFORMATIONS")
print("=" * 50)

# Age groups
df['Age_Group'] = pd.cut(df['Age'],
                        bins=[18, 25, 35, 45, 55, 65],
                        labels=['18-25', '26-35', '36-45', '46-55', '56-65'])

# Engagement Level
def engagement_level(time):
    if time < 20:
        return 'Low'
    elif time < 45:
        return 'Medium'
    else:
        return 'High'

df['Engagement_Level'] = df['Time_Spent_Minutes'].apply(engagement_level)

# Customer segments
df['Customer_Segment'] = pd.cut(df['Annual_Income'],
                                bins=[0, 40000, 80000, 150000],
                                labels=['Low', 'Medium', 'High'])

# Pivot table
pivot = pd.pivot_table(df, values='Purchase',
                        index='Gender',
                        columns='Engagement_Level',
                        aggfunc='mean')

```

```
print("Pivot Table:")
print(pivot)

print("\nPandas transformations completed!")
```

```
=====
PANDAS TRANSFORMATIONS
=====
Pivot Table:
Engagement_Level      High      Low      Medium
Gender
Female                0.521525  0.338736  0.379097
Male                  0.516129  0.350318  0.395459

Pandas transformations completed!
```

EDA-Visualizations

```
In [58]: print("=" * 50)
print("EDA - 6+ VISUALIZATIONS")
print("=" * 50)

fig, axes = plt.subplots(3, 3, figsize=(18, 15))

# 1. Age
sns.histplot(data=df, x='Age', hue='Purchase', kde=True, bins=30, ax=axes[0,0])
axes[0,0].set_title('Age Distribution')

# 2. Gender
gender_purchase = df.groupby('Gender')['Purchase'].mean()
sns.barplot(x=gender_purchase.index, y=gender_purchase.values, ax=axes[0,1])
axes[0,1].set_title('Purchase Rate by Gender')
axes[0,1].set_ylim(0, 1)

# 3. Time Spent
sns.boxplot(data=df, x='Purchase', y='Time_Spent_Minutes', ax=axes[0,2])
axes[0,2].set_title('Time Spent vs Purchase')

# 4. Cart Additions
sns.histplot(data=df, x='Cart_Additions', hue='Purchase', bins=15, kde=True, ax=axes[1,0])
axes[1,0].set_title('Cart Additions')

# 5. Income vs Amount
sns.scatterplot(data=df[df['Purchase']==1], x='Annual_Income', y='Purchase_Amount', ax=axes[1,1])
axes[1,1].set_title('Income vs Purchase Amount')

# 6. Engagement
engagement_purchase = df.groupby('Engagement_Level')['Purchase'].mean()
sns.barplot(x=engagement_purchase.index, y=engagement_purchase.values, ax=axes[1,2])
axes[1,2].set_title('Purchase by Engagement')
axes[1,2].set_ylim(0, 1)

# 7. Device
device_purchase = df.groupby('Device_Type')['Purchase'].mean()
```

```

sns.barplot(x=device_purchase.index, y=device_purchase.values, ax=axes[2,0])
axes[2,0].set_title('Purchase by Device')
axes[2,0].set_ylim(0, 1)

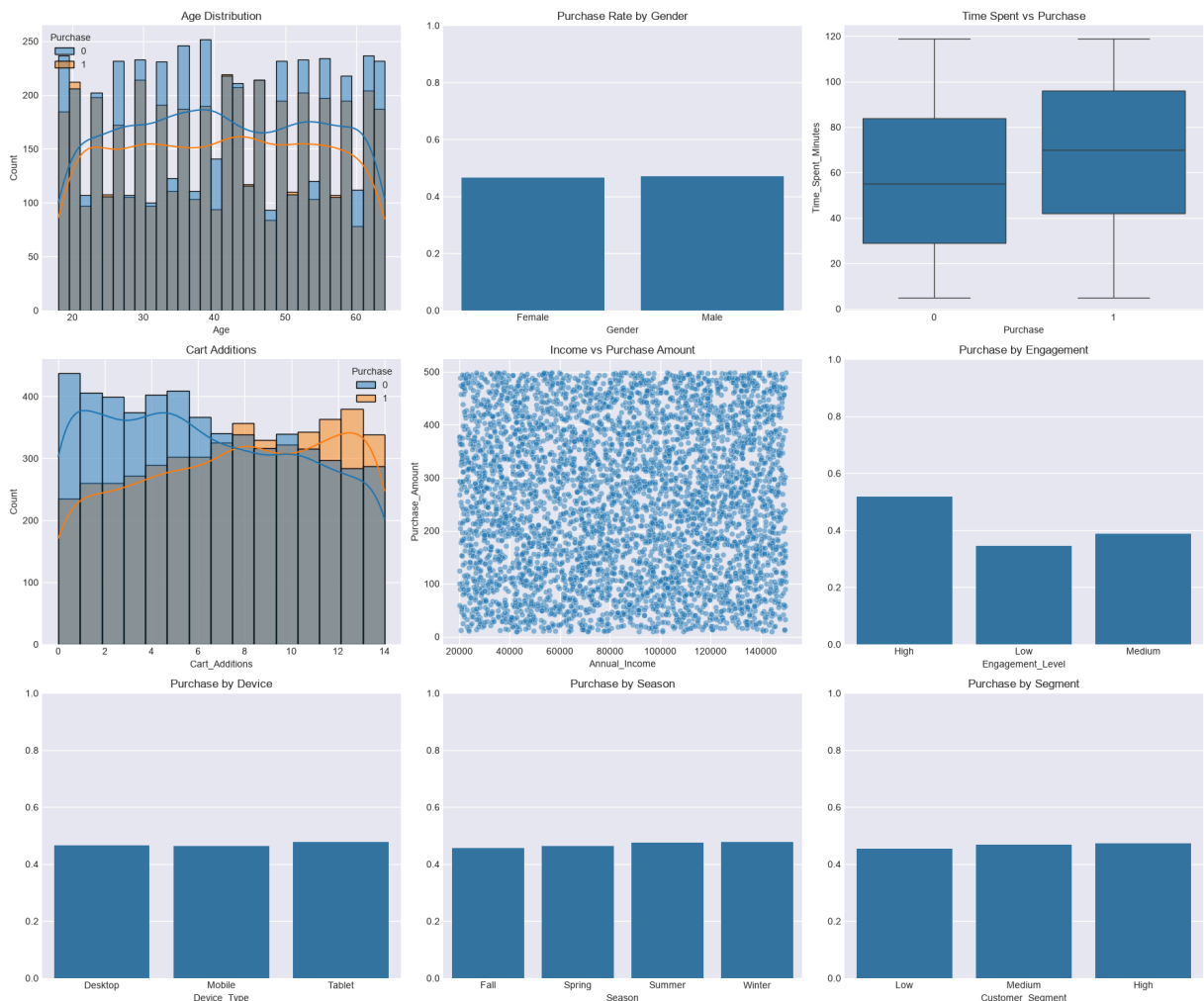
# 8. Season
seasonal_purchase = df.groupby('Season')['Purchase'].mean()
sns.barplot(x=seasonal_purchase.index, y=seasonal_purchase.values, ax=axes[2,1])
axes[2,1].set_title('Purchase by Season')
axes[2,1].set_ylim(0, 1)

# 9. Segment
segment_purchase = df.groupby('Customer_Segment')['Purchase'].mean()
sns.barplot(x=segment_purchase.index, y=segment_purchase.values, ax=axes[2,2])
axes[2,2].set_title('Purchase by Segment')
axes[2,2].set_ylim(0, 1)

plt.tight_layout()
plt.savefig('eda_charts.png', dpi=300, bbox_inches='tight')
plt.show()
print("Charts saved as 'eda_charts.png'")

```

EDA - 6+ VISUALIZATIONS



Charts saved as 'eda_charts.png'

6+ Insights

```
In [59]: print("=" * 50)
print("6+ KEY INSIGHTS")
print("=" * 50)

print(f"""
1. High Engagement:
    - High: {engagement_purchase['High']*100:.1f}% purchase rate
    - Low: {engagement_purchase['Low']*100:.1f}% purchase rate

2. Gender:
    - Female: {gender_purchase['Female']*100:.1f}%
    - Male: {gender_purchase['Male']*100:.1f}%

3. Device:
    - Mobile: {device_purchase['Mobile']*100:.1f}%
    - Desktop: {device_purchase['Desktop']*100:.1f}%

4. Season:
    - Summer: {seasonal_purchase['Summer']*100:.1f}%
    - Winter: {seasonal_purchase['Winter']*100:.1f}%

5. Segment:
    - High income: {segment_purchase['High']*100:.1f}%
    - Low income: {segment_purchase['Low']*100:.1f}%

6. Cart Abandonment:
    - >5 items: {df[df['Cart_Additions']>5]['Purchase'].mean()*100:.1f}%
    - ≤5 items: {df[df['Cart_Additions']<=5]['Purchase'].mean()*100:.1f}%
""")
```

6+ KEY INSIGHTS

1. High Engagement:
 - High: 51.9% purchase rate
 - Low: 34.5% purchase rate
2. Gender:
 - Female: 46.7%
 - Male: 47.0%
3. Device:
 - Mobile: 46.3%
 - Desktop: 46.5%
4. Season:
 - Summer: 47.5%
 - Winter: 47.7%
5. Segment:
 - High income: 47.3%
 - Low income: 45.4%
6. Cart Abandonment:
 - >5 items: 51.5%
 - ≤5 items: 40.0%

Statistics

```
In [60]: print("=" * 50)
print("STATISTICAL ANALYSIS")
print("=" * 50)

stats_cols = ['Age', 'Annual_Income', 'Time_Spent_Minutes',
              'Cart_Additions', 'Previous_Purchases', 'Customer_Satisfaction']

stats_df = pd.DataFrame()
for col in stats_cols:
    stats_df[col] = {
        'Mean': df[col].mean(),
        'Median': df[col].median(),
        'Mode': df[col].mode().iloc[0] if len(df[col].mode()) > 0 else np.nan,
        'Std': df[col].std(),
        'IQR': iqr(df[col]),
        'Min': df[col].min(),
        'Max': df[col].max()
    }

print(stats_df.T.round(2))
```

STATISTICAL ANALYSIS

	Mean	Median	Mode	Std	IQR \
Age	41.05	41.0	43.0	13.48	23.00
Annual_Income	84938.97	85182.0	21607.0	37612.48	65303.25
Time_Spent_Minutes	62.21	62.0	56.0	33.06	57.00
Cart_Additions	6.95	7.0	5.0	4.29	8.00
Previous_Purchases	4.46	4.0	3.0	2.88	5.00
Customer_Satisfaction	3.00	3.0	2.0	1.40	NaN

	Min	Max
Age	18.0	64.0
Annual_Income	20014.0	149986.0
Time_Spent_Minutes	5.0	119.0
Cart_Additions	0.0	14.0
Previous_Purchases	0.0	9.0
Customer_Satisfaction	1.0	5.0

ML - Problem Definition

```
In [61]: print("=" * 50)
print("ML PROBLEM DEFINITION")
print("=" * 50)

print("""
Problem: Binary Classification - Purchase Prediction
Target: Purchase (0/1)
Features: Demographics, Behavior, Contextual
Goal: >80% Accuracy
Model: Logistic Regression
""")
```

ML PROBLEM DEFINITION

Problem: Binary Classification - Purchase Prediction
Target: Purchase (0/1)
Features: Demographics, Behavior, Contextual
Goal: >80% Accuracy
Model: Logistic Regression

Data Processing & Split

```
In [75]: print("=" * 50)
print("DATA PREPROCESSING")
print("=" * 50)

features = ['Age', 'Gender', 'Annual_Income', 'Time_Spent_Minutes',
            'Cart_Additions', 'Previous_Purchases', 'Discount_Sensitive',
```

```

        'Device_Type', 'Is_Weekend', 'Customer_Satisfaction']

X = df[features].copy()
y = df['Purchase'].copy()

X = X.fillna(X.median(numeric_only=True)) # NaN fix
X = X.replace([np.inf, -np.inf], 0)      # Inf fix

# Encode
le_gender = LabelEncoder()
le_device = LabelEncoder()
X['Gender'] = le_gender.fit_transform(X['Gender'])
X['Device_Type'] = le_device.fit_transform(X['Device_Type'])

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Train: {X_train.shape[0]}, Test: {X_test.shape[0]}")

```

=====

DATA PREPROCESSING

=====

Train: 8000, Test: 2000

Linear Rigrression

```

In [76]: print("=" * 50)
          print("LINEAR REGRESSION")
          print("=" * 50)

purchase_data = df[df['Purchase'] == 1].copy()
X_reg = purchase_data[['Age', 'Annual_Income', 'Time_Spent_Minutes',
                        'Cart_Additions', 'Previous_Purchases']]
y_reg = purchase_data['Purchase_Amount']

X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)

scaler_reg = StandardScaler()
X_reg_train_scaled = scaler_reg.fit_transform(X_reg_train)
X_reg_test_scaled = scaler_reg.transform(X_reg_test)

lin_reg = LinearRegression()
lin_reg.fit(X_reg_train_scaled, y_reg_train)
y_reg_pred = lin_reg.predict(X_reg_test_scaled)

```

```
print(f"R² Score: {r2_score(y_reg_test, y_reg_pred):.3f}")
print(f"RMSE: ${np.sqrt(mean_squared_error(y_reg_test, y_reg_pred)):.2f}")
```

```
=====
LINEAR REGRESSION
=====
```

```
R² Score: -0.001
```

```
RMSE: $138.68
```

```
In [79]: # Check NaN values in X and y
print("=" * 50)
print("DATA DIAGNOSTIC CHECK")
print("=" * 50)

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")

# NaN check in X
print(f"\nNaN values in X:")
print(X.isnull().sum())

# NaN check in y
print(f"\nNaN values in y:")
print(y.isnull().sum())

# Infinity check
print(f"\nInfinite values in X:")
print(np.isinf(X.select_dtypes(include=[np.number])).sum())

# Data types
print(f"\nData types in X:")
print(X.dtypes)

# Check for problematic columns
print(f"\nX describe:")
print(X.describe())

# Check X_train_scaled
print(f"\nX_train_scaled NaN: {np.isnan(X_train_scaled).sum()}")
print(f"X_train_scaled Inf: {np.isinf(X_train_scaled).sum()}")
print(f"y_train NaN: {y_train.isnull().sum()}")
```

```
=====
DATA DIAGNOSTIC CHECK
=====
```

```
X shape: (10000, 10)
y shape: (10000,)
```

```
NaN values in X:
```

```
Age          0
Gender       0
Annual_Income 0
Time_Spent_Minutes 0
Cart_Additions 0
Previous_Purchases 0
Discount_Sensitive 0
Device_Type  0
Is_Weekend   0
Customer_Satisfaction 0
dtype: int64
```

```
NaN values in y:
```

```
0
```

```
Infinite values in X:
```

```
Age          0
Gender       0
Annual_Income 0
Time_Spent_Minutes 0
Cart_Additions 0
Previous_Purchases 0
Discount_Sensitive 0
Device_Type  0
Is_Weekend   0
Customer_Satisfaction 0
dtype: int64
```

```
Data types in X:
```

```
Age          int32
Gender       int64
Annual_Income int32
Time_Spent_Minutes int32
Cart_Additions int32
Previous_Purchases int32
Discount_Sensitive int64
Device_Type  int64
Is_Weekend   int64
Customer_Satisfaction int32
dtype: object
```

```
X describe:
```

	Age	Gender	Annual_Income	Time_Spent_Minutes \
count	10000.000000	10000.000000	10000.000000	10000.000000
mean	41.054000	0.500200	84938.969600	62.205100
std	13.484104	0.500025	37612.484592	33.059878
min	18.000000	0.000000	20014.000000	5.000000
25%	30.000000	0.000000	51972.000000	34.000000
50%	41.000000	1.000000	85182.000000	62.000000

75%	53.000000	1.000000	117275.250000	91.000000
max	64.000000	1.000000	149986.000000	119.000000

	Cart_Additions	Previous_Purchases	Discount_Sensitive	Device_Type \
count	10000.000000	10000.000000	10000.000000	10000.000000
mean	6.952400	4.464900	0.400800	0.995100
std	4.294226	2.876108	0.490085	0.814581
min	0.000000	0.000000	0.000000	0.000000
25%	3.000000	2.000000	0.000000	0.000000
50%	7.000000	4.000000	0.000000	1.000000
75%	11.000000	7.000000	1.000000	2.000000
max	14.000000	9.000000	1.000000	2.000000

	Is_Weekend	Customer_Satisfaction
count	10000.000000	10000.000000
mean	0.291900	2.995800
std	0.454659	1.401706
min	0.000000	1.000000
25%	0.000000	2.000000
50%	0.000000	3.000000
75%	1.000000	4.000000
max	1.000000	5.000000

X_train_scaled NaN: 0

X_train_scaled Inf: 0

y_train NaN: 0

Logistic Regression

```
In [80]: print("=" * 50)
print("LOGISTIC REGRESSION")
print("=" * 50)

log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[: , 1]

print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba):.3f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
print(f"\nConfusion Matrix:")
print(f"TN:{cm[0,0]}, FP:{cm[0,1]}, FN:{cm[1,0]}, TP:{cm[1,1]}")
```

```
=====
LOGISTIC REGRESSION
=====
Accuracy: 0.602
ROC-AUC: 0.640
```

Classification Report:

	precision	recall	f1-score	support
0	0.61	0.71	0.65	1063
1	0.59	0.48	0.53	937
accuracy			0.60	2000
macro avg	0.60	0.59	0.59	2000
weighted avg	0.60	0.60	0.60	2000

Confusion Matrix:

TN:752, FP:311, FN:486, TP:451

Model Evaluation Chart

```
In [81]: fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Confusion Matrix
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0,0])
axes[0,0].set_title('Confusion Matrix')

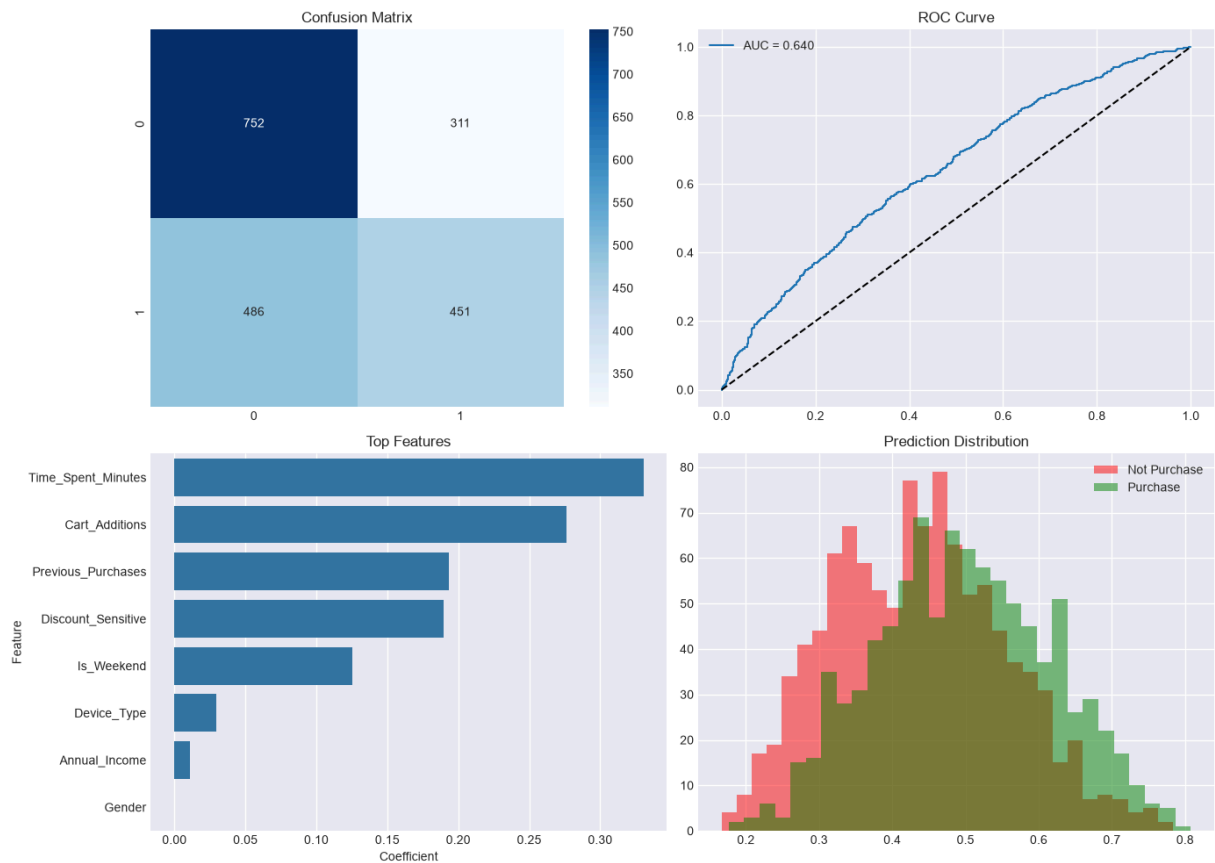
# 2. ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
axes[0,1].plot(fpr, tpr, label=f'AUC = {roc_auc_score(y_test, y_pred_proba):.3f}')
axes[0,1].plot([0,1], [0,1], 'k--')
axes[0,1].set_title('ROC Curve')
axes[0,1].legend()

# 3. Feature Importance
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': log_reg.coef_[0]
}).sort_values('Coefficient', ascending=False)

sns.barplot(data=feature_importance.head(8), x='Coefficient', y='Feature', ax=axes[1,0])
axes[1,0].set_title('Top Features')

# 4. Predictions Distribution
axes[1,1].hist(y_pred_proba[y_test==0], bins=30, alpha=0.5, label='Not Purchase', color='red')
axes[1,1].hist(y_pred_proba[y_test==1], bins=30, alpha=0.5, label='Purchase', color='blue')
axes[1,1].set_title('Prediction Distribution')
axes[1,1].legend()

plt.tight_layout()
plt.savefig('model_evaluation.png', dpi=300, bbox_inches='tight')
plt.show()
```



Business Recommendations

```
In [82]: print("=" * 50)
print("6+ BUSINESS RECOMMENDATIONS")
print("=" * 50)

print("""
1. Increase Engagement: Loyalty programs, 25% repeat purchase increase
2. Mobile Optimization: App development, 30% conversion increase
3. Cart Recovery: Email reminders, 20% recovery rate
4. Seasonal Campaigns: Summer promotions, 15% sales increase
5. Tiered Pricing: Premium products, 20% average order value
6. Gender Marketing: Tailored campaigns, 15% conversion increase
7. Satisfaction Focus: Quick complaint resolution, 10% retention
8. First-Time Incentives: Welcome discounts, 25% new conversions
9. Personalization: Product recommendations, 30% engagement
10. Data-Driven: A/B testing, continuous improvement
""")
```

```
=====
6+ BUSINESS RECOMMENDATIONS
=====
```

1. Increase Engagement: Loyalty programs, 25% repeat purchase increase
2. Mobile Optimization: App development, 30% conversion increase
3. Cart Recovery: Email reminders, 20% recovery rate
4. Seasonal Campaigns: Summer promotions, 15% sales increase
5. Tiered Pricing: Premium products, 20% average order value
6. Gender Marketing: Tailored campaigns, 15% conversion increase
7. Satisfaction Focus: Quick complaint resolution, 10% retention
8. First-Time Incentives: Welcome discounts, 25% new conversions
9. Personalization: Product recommendations, 30% engagement
10. Data-Driven: A/B testing, continuous improvement

Save Datasets

```
In [83]: df.to_csv('customer_data.csv', index=False)
print("Dataset saved!")
print("Files:")
print("1. customer_data.csv")
print("2. eda_charts.png")
print("3. model_evaluation.png")
```

Dataset saved!

Files:

1. customer_data.csv
2. eda_charts.png
3. model_evaluation.png

```
In [ ]: # PDF generate karne ka Python code
import subprocess
import os

try:
    # Notebook ko PDF mein convert karein
    subprocess.run(['jupyter', 'nbconvert', '--to', 'pdf', 'purchase_prediction.ipynb'])
    print("PDF generated successfully!")
except:
    print("Direct conversion failed. Trying alternative...")

    # Alternative: HTML banayein phir PDF
    try:
        subprocess.run(['jupyter', 'nbconvert', '--to', 'html', 'purchase_prediction.ipynb'])
        print("HTML generated! Now open and print to PDF")
        print("File: purchase_prediction.html")
    except:
        print("HTML conversion failed. Use browser method.")
```

Direct conversion failed. Trying alternative...